

Application Layer

Definition

► The **Application Layer** is the topmost layer of the TCP/IP protocol suite. It provides services directly to users and application programs and enables communication between applications running on different hosts.

1. The Application Layer provides services to the user.
2. Communication is achieved through a **logical connection** between application layers on different hosts.
3. The two application layers behave as if there is a direct connection between them.
4. In reality, data passes through all lower layers before reaching the destination.
5. Communication at this layer is **process-to-process communication**.
6. A process sends a **request** to another process and receives a **response**.

Logical Communication

- The Application Layer at the sender communicates logically with the Application Layer at the receiver.
- This communication is called **logical communication** because the connection is imaginary.
- Actual data transfer occurs through the Transport, Network, Data Link, and Physical layers.

Example

► When a user opens a website:

1. The web browser sends an HTTP request.
2. The request travels through all TCP/IP layers.
3. The destination web server receives the request.
4. The server sends an HTTP response back to the browser.
5. To the applications, it appears as if they are communicating directly.

Responsibilities of the Application Layer

- ▶ Providing network services to users
- ▶ Process-to-process communication
- ▶ Request and response handling
- ▶ Data formatting and representation
- ▶ Resource sharing

Common Application Layer Protocols

- ▶ **HTTP** – Web browsing
- ▶ **HTTPS** – Secure web communication
- ▶ **FTP** – File transfer
- ▶ **SMTP** – Sending emails
- ▶ **DNS** – Domain name resolution
- ▶ **Telnet** – Remote login
- ▶ **SSH** – Secure remote access
- ▶ **SNMP** – Network management

Application-Layer Paradigms

1. Internet communication requires two application programs running on different computers.
2. These programs exchange messages through the Internet.
3. The relationship between the communicating programs is defined by an **Application-Layer Paradigm**.
4. An application program may:
 - Request a service
 - Provide a service
 - Or perform both functions
5. Two major paradigms have evolved for application-layer communication:
 - **Client-Server Paradigm**
 - **Peer-to-Peer (P2P) Paradigm**

Traditional Paradigm: Client–Server

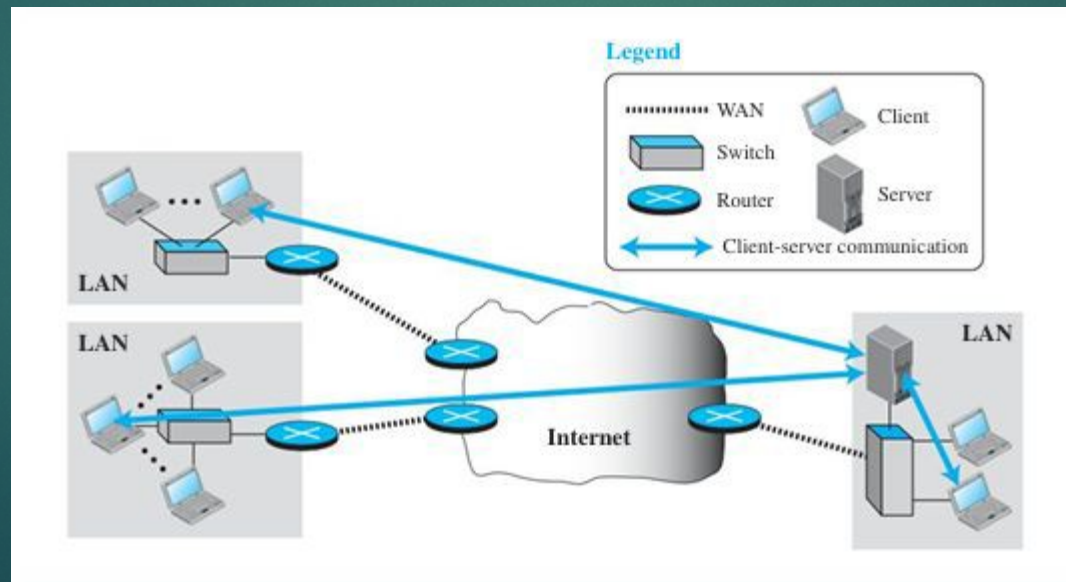
- ▶ The **Client–Server Paradigm** is a communication model in which a **server process** provides services and a **client process** requests those services through the Internet.
- 1. The server is the service provider.
- 2. The client requests services from the server.
- 3. The server runs continuously and waits for client requests.
- 4. The client starts only when a service is needed.
- 5. Many clients can connect to a server.
- 6. The roles of the client and server are different and can operate independently.
- 7. Separate client and server programs are required for communication between the client and the server.

Working of Client–Server Model

1. The client initiates a connection to the server.
2. The client sends a service request.
3. The server processes the request.
4. The server sends a response to the client.
5. The client receives the requested service.

Real-Life Example

- ▶ **Telephone Directory Service**
 - **Directory Center** → Server
 - **Subscriber** → Client
- ▶ The directory center is always available, while the subscriber contacts it only when information is needed.



Advantages

1. Centralized control and management.
2. Easy maintenance and updates.
3. Efficient resource sharing.
4. Better security and data management.

Disadvantages

1. Heavy workload on the server.
2. Requires powerful server hardware.
3. Server failure affects all connected clients.
4. High setup and maintenance costs.
5. Performance may degrade when many clients connect simultaneously.

Examples of Client–Server Applications

- WWW (World Wide Web)
- HTTP (HyperText Transfer Protocol)
- FTP (File Transfer Protocol)
- SSH (Secure Shell)
- E-mail Services

New Paradigm: Peer-to-Peer (P2P)

► The **Peer-to-Peer (P2P) Paradigm** is a communication model in which every computer (peer) can act as both a **service provider** and a **service requester**. There is no need for a dedicated server running all the time.

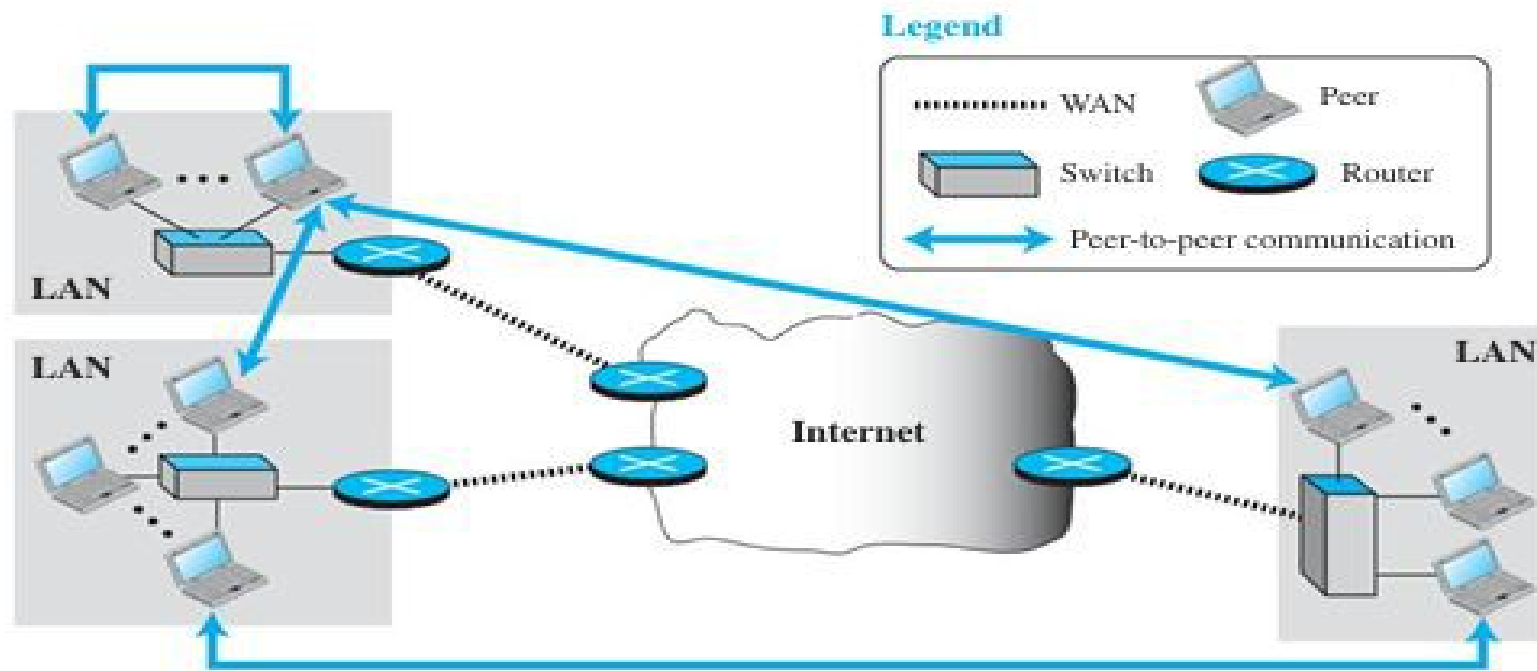
1. No dedicated server is required.
2. Responsibility is shared among all peers.
3. A peer can provide services to other peers.
4. A peer can receive services from other peers.
5. A peer can provide and receive services simultaneously.
6. Each computer has equal status in the network.
7. The workload is distributed among multiple peers.

Working of P2P Model

1. A peer requests a service from another peer.
2. The receiving peer provides the requested service.
3. The roles can change at any time.
4. The same peer can act as both client and server.

Working of P2P Model

Figure 2.3 Example of a peer-to-peer paradigm



Advantages

1. No need for expensive dedicated servers.
2. Cost-effective.
3. Easily scalable.
4. Workload is distributed among peers.
5. Better utilization of network resources.

Disadvantages

1. Security is more challenging.
2. Difficult to manage distributed services.
3. Data availability depends on participating peers.
4. Not suitable for all Internet applications.

Applications of P2P

- Internet Telephony
- File Sharing
- BitTorrent
- IPTV
- Voice and Video Communication

Example

► Internet Telephony

- Both users communicate directly.
- No dedicated server is required to continuously provide the service.
- Each participant acts as a peer.

► File Sharing

- Users share files directly with each other.
- A user does not need to maintain a permanent server.

Mixed Paradigm

► The **Mixed Paradigm** combines the features of both the **Client–Server** and **Peer-to-Peer (P2P)** paradigms to take advantage of the strengths of each model.

1. Combines **Client–Server** and **Peer-to-Peer** communication.
2. A central server is used only for specific tasks such as locating peers.
3. The actual service or data transfer occurs directly between peers.
4. Reduces the load on the server.
5. Improves scalability and efficiency.
6. Utilizes the advantages of both paradigms.

Working of Mixed Paradigm

1. A client contacts a server to find the address of a peer.
2. The server provides the peer's address.
3. The client establishes a direct connection with the peer.
4. Data or services are exchanged directly between peers.
5. The server is no longer involved in the actual communication.

Advantages

1. Reduces server workload.
2. More scalable than pure client-server architecture.
3. Cost-effective.
4. Efficient resource sharing.
5. Combines centralized control with distributed communication.

Applications

- Peer-to-Peer File Sharing
- Internet Telephony
- Skype
- BitTorrent
- IPTV

Client–Server Paradigm

► The **Client–Server Paradigm** is a communication model in which a **client** sends a request for a service, and a **server** waits for requests, processes them, and sends back a response.

1. Client

- Initiates communication.
- Requests services from the server.
- Runs only when needed.

2. Server

- Provides services to clients.
- Runs continuously and waits for requests.
- Can serve multiple clients.

3. The **server's lifetime is infinite**, while the **client's lifetime is temporary**

Application Programming Interface (API)

- ▶ An API (**A**pplication **P**rogramming **I**nterface) is a set of instructions that allows application programs to communicate with the operating system and the TCP/IP protocol suite.

Functions of API

- Open a connection
- Send data
- Receive data
- Close a connection

Common APIs

- Socket Interface
- Transport Layer Interface (TLI)
- STREAM

Socket

► A **Socket** is an abstraction that acts as an endpoint for communication between a client process and a server process.

1. A socket is created by an application program.
2. Communication occurs between two sockets.
3. The client sends requests through its socket.
4. The server sends responses through its socket.
5. Sockets act as both source and destination of data.

Socket Address

- ▶ A Socket Address uniquely identifies a process on a computer.
- ▶ Components
 1. IP Address (32 bits)
 - Identifies the computer.
 2. Port Number (16 bits)
 - Identifies the specific application process.

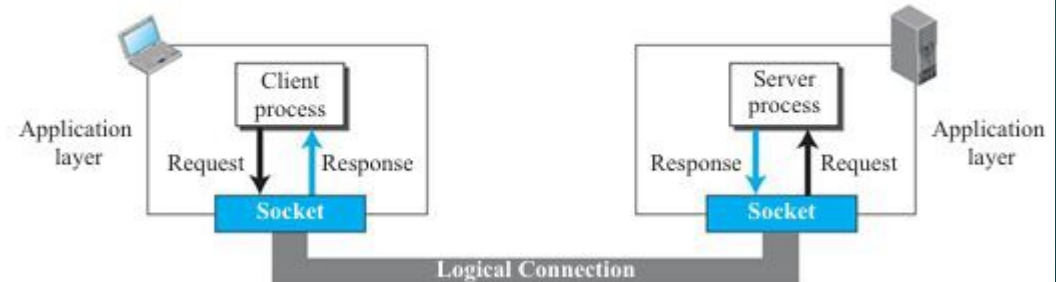
Socket Address Formula

- *Socket Address = IP Address + Port Number*

Figure 2.7 A socket address



Figure 2.6 Use of sockets in process-to-process communication



Finding Socket Addresses

Server Side

▶ Local Socket Address

- IP address is known by the operating system.
- Port number is fixed and assigned to the server.
- Example: HTTP uses Port 80.

▶ Remote Socket Address

- Obtained from the client's request.
- Changes for each client connection.

Finding Socket Addresses

Client Side

▶ Local Socket Address

- IP address is assigned by the operating system.
- A temporary port number is assigned for communication.

▶ Remote Socket Address

- Consists of the server's IP address and port number.
- Can be obtained directly or through DNS.

Using Services of the Transport Layer

1. Application processes use the services provided by the **Transport Layer** for communication.
2. The TCP/IP protocol suite provides three common transport-layer protocols:
 - **UDP (User Datagram Protocol)**
 - **TCP (Transmission Control Protocol)**
 - **SCTP (Stream Control Transmission Protocol)**
3. The choice of transport protocol affects the performance and capabilities of an application.

Using Services of the Transport Layer

► UDP (User Datagram Protocol) Characteristics

- Connectionless protocol
- Unreliable communication
- Message-oriented
- Fast and simple
- No flow control or congestion control

Advantages

- Low overhead
- Faster transmission
- Suitable for small messages

Applications

- Multimedia streaming
- Network management
- VoIP applications

Example

- ▶ Similar to sending letters through the post office.
- ▶ Each message is independent.

TCP (Transmission Control Protocol)

► Characteristics

- Connection-oriented protocol
- Reliable communication
- Byte-stream oriented
- Provides flow control
- Provides error control
- Provides congestion control

Advantages

- Reliable data delivery
- Lost or corrupted data can be retransmitted
- Suitable for large data transfers

Applications

- Web browsing (HTTP/HTTPS)
- File Transfer (FTP)
- E-mail services

▶ **Example**

- ▶ Similar to a telephone conversation where communication is established before exchanging information.

SCTP (Stream Control Transmission Protocol)

► Characteristics

- Connection-oriented protocol
- Reliable communication
- Message-oriented
- Supports multistream communication
- Can maintain communication even if one path fails

Advantages

- Combines features of TCP and UDP
- Reliable and message-oriented
- Better fault tolerance

Applications

- Multimedia communication
- Telecommunication systems
- Applications requiring high reliability